

Homework Sorting Project

組員:41343117 林晉邦、

41343151 鍾定志

解題說明:

本題要求設計一個綜合排序程式，將 **Insertion Sort**、**Quick Sort**、**Merge Sort** 和 **Heap Sort** 四種排序法整合在同一個程式中。程式需針對不同資料量產生測試資料，測量各排序法的執行時間，並依照測試結果設計 **Composite Sort**，使程式能根據資料量選擇較合適的排序方式。

解題策略：

本題先分別完成四種排序演算法，再建立不同資料量的測試資料進行排序與計時。透過比較各排序法在不同 n 值下的執行效率，觀察哪一種排序法在特定範圍內較適合使用。最後根據測試結果設計 **Composite Sort**，讓程式能依資料量自動選擇較有效率的排序法。

程式碼

```
Insertion Sort.cpp X
C: > Users > tusno > Downloads > Insertion Sort.cpp > main()

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  #include <random>
5  #include <chrono>
6  #include <iomanip>
7  #include <cstdlib>
8  #include <ctime>
9
10 using namespace std;
11 using namespace chrono;
12
13 bool isSorted(vector<int> arr)
14 {
15     for (int i = 0; i < arr.size(); i++)
16     {
17         for (int j = i + 1; j < arr.size(); j++)
18         {
19             if (arr[i] > arr[j])
20             {
21                 return false;
22             }
23         }
24     }
25
26     return true;
27 }
28
29 void insertionSort(vector<int>& arr)
30 {
31     int n = arr.size();
32
33     for (int i = 1; i < n; i++)
34     {
35         int temp = arr[i];
36         int j = i - 1;
37
38         while (j >= 0)
39         {
40             if (arr[j] > temp)
41             {
42                 arr[j + 1] = arr[j];
43                 j--;
44             }
45             else
46             {
47                 break;
48             }
49         }
50         arr[j + 1] = temp;
51     }
52 }
```

```

48     }
49 }
50
51     arr[j + 1] = temp;
52 }
53 }
54
55 void insertionSortRange(vector<int>& arr, int left, int right)
56 {
57     for (int i = left + 1; i <= right; i++)
58     {
59         int temp = arr[i];
60         int j = i - 1;
61
62         while (j >= left && arr[j] > temp)
63         {
64             arr[j + 1] = arr[j];
65             j--;
66         }
67
68         arr[j + 1] = temp;
69     }
70 }
71
72 int medianOfThree(vector<int>& arr, int left, int right)
73 {
74     int mid = (left + right) / 2;
75
76     if (arr[left] > arr[mid])
77     {
78         swap(arr[left], arr[mid]);
79     }
80
81     if (arr[left] > arr[right])
82     {
83         swap(arr[left], arr[right]);
84     }
85
86     if (arr[mid] > arr[right])
87     {
88         swap(arr[mid], arr[right]);

```

按一下以新增中斷點

```
91     swap(arr[mid], arr[right - 1]);
92
93     return arr[right - 1];
94 }
95
96 void quickSortRecursive(vector<int>& arr, int left, int right)
97 {
98     if (left >= right)
99     {
100         return;
101     }
102
103     if (right - left <= 20)
104     {
105         insertionSortRange(arr, left, right);
106         return;
107     }
108
109     int pivot = medianOfThree(arr, left, right);
110
111     int i = left;
112     int j = right - 1;
113
114     while (true)
115     {
116         while (arr[++i] < pivot)
117         {
118
119         }
120
121         while (arr[--j] > pivot)
122         {
123
124         }
125
126         if (i < j)
127         {
128             swap(arr[i], arr[j]);
129         }
130         else
131         {
132             break;
133         }
134     }
135 }
```

```
134     swap(arr[i], arr[right - 1]);
135
136     quickSortRecursive(arr, left, i - 1);
137     quickSortRecursive(arr, i + 1, right);
138 }
139
140 void quickSort(vector<int>& arr)
141 {
142     if (arr.size() <= 1)
143     {
144         return;
145     }
146
147     quickSortRecursive(arr, 0, arr.size() - 1);
148 }
149
150 void mergeParts(vector<int>& arr, int left, int mid, int right)
151 {
152     vector<int> leftArray;
153     vector<int> rightArray;
154
155     for (int i = left; i <= mid; i++)
156     {
157         leftArray.push_back(arr[i]);
158     }
159
160     for (int i = mid + 1; i <= right; i++)
161     {
162         rightArray.push_back(arr[i]);
163     }
164
165     int i = 0;
166     int j = 0;
167     int k = left;
168
169     while (i < leftArray.size() && j < rightArray.size())
170     {
171         if (leftArray[i] <= rightArray[j])
172         {
173             arr[k] = leftArray[i];
174             i++;
175         }
176         else
177         {
```

```

178         arr[k] = rightArray[j];
179         j++;
180     }
181
182     k++;
183 }
184
185 while (i < leftArray.size())
186 {
187     arr[k] = leftArray[i];
188     i++;
189     k++;
190 }
191
192 while (j < rightArray.size())
193 {
194     arr[k] = rightArray[j];
195     j++;
196     k++;
197 }
198 }
199
200 void mergeSort(vector<int>& arr)
201 {
202     int n = arr.size();
203
204     int size = 1;
205
206     while (size < n)
207     {
208         int left = 0;
209
210         while (left < n - 1)
211         {
212             int mid = left + size - 1;
213             int right = left + 2 * size - 1;
214
215             if (mid >= n)
216             {
217                 mid = n - 1;
218             }

```



```
220         if (right >= n)
221         {
222             right = n - 1;
223         }
224
225         if (mid < right)
226         {
227             mergeParts(arr, left, mid, right);
228         }
229
230         left = left + 2 * size;
231     }
232
233     size = size * 2;
234 }
235 }
236
237 void heapify(vector<int>& arr, int n, int i)
238 {
239     int largest = i;
240
241     int left = 2 * i + 1;
242     int right = 2 * i + 2;
243
244     if (left < n)
245     {
246         if (arr[left] > arr[largest])
247         {
248             largest = left;
249         }
250     }
251
252     if (right < n)
253     {
254         if (arr[right] > arr[largest])
255         {
256             largest = right;
257         }
258     }
259
260     if (largest != i)
261     {
262         int temp = arr[i];
```

```

262     int temp = arr[i];
263     arr[i] = arr[largest];
264     arr[largest] = temp;
265
266     heapify(arr, n, largest);
267 }
268 }
269
270 void heapSort(vector<int>& arr)
271 {
272     int n = arr.size();
273
274     for (int i = n / 2 - 1; i >= 0; i--)
275     {
276         heapify(arr, n, i);
277     }
278
279     for (int i = n - 1; i >= 1; i--)
280     {
281         int temp = arr[0];
282         arr[0] = arr[i];
283         arr[i] = temp;
284
285         heapify(arr, i, 0);
286     }
287 }
288
289 vector<int> generateSortedData(int n)
290 {
291     vector<int> arr;
292
293     for (int i = 1; i <= n; i++)
294     {
295         arr.push_back(i);
296     }
297
298     return arr;
299 }
300
301 vector<int> generateReverseData(int n)
302 {
303     vector<int> arr;
304
305     for (int i = n; i >= 1; i--)

```

```
305     for (int i = n; i >= 1; i--)
306     {
307         arr.push_back(i);
308     }
309
310     return arr;
311 }
312
313 vector<int> generateRandomPermutation(int n)
314 {
315     vector<int> arr;
316
317     for (int i = 1; i <= n; i++)
318     {
319         arr.push_back(i);
320     }
321
322     for (int i = n - 1; i >= 1; i--)
323     {
324         int j = rand() % (i + 1);
325
326         int temp = arr[i];
327         arr[i] = arr[j];
328         arr[j] = temp;
329     }
330
331     return arr;
332 }
333
334 vector<int> generateMergeWorstCase(vector<int> arr)
335 {
336     int n = arr.size();
337
338     if (n <= 1)
339     {
340         return arr;
341     }
342
343     vector<int> left;
344     vector<int> right;
345
346     for (int i = 0; i < n; i++)
347     {
348         if (i % 2 == 0)
349         {
```

```

348         if (i % 2 == 0)
349         {
350             left.push_back(arr[i]);
351         }
352         else
353         {
354             right.push_back(arr[i]);
355         }
356     }
357
358     left = generateMergeWorstCase(left);
359     right = generateMergeWorstCase(right);
360
361     vector<int> result;
362
363     for (int i = 0; i < left.size(); i++)
364     {
365         result.push_back(left[i]);
366     }
367
368     for (int i = 0; i < right.size(); i++)
369     {
370         result.push_back(right[i]);
371     }
372
373     return result;
374 }
375
376 vector<int> generateMergeWorstCaseData(int n)
377 {
378     vector<int> arr = generateSortedData(n);
379     vector<int> result = generateMergeWorstCase(arr);
380
381     return result;
382 }
383
384 double measureTime(void (*sortFunction)(vector<int>&), vector<int> data, int repeatTimes)
385 {
386     double total = 0.0;
387
388     for (int i = 0; i < repeatTimes; i++)
389     {
390         vector<int> copyData;

```

```

392     for (int j = 0; j < data.size(); j++)
393     {
394         copyData.push_back(data[j]);
395     }
396
397     auto start = high_resolution_clock::now();
398
399     sortFunction(copyData);
400
401     auto end = high_resolution_clock::now();
402
403     duration<double, milli> elapsed = end - start;
404
405     total = total + elapsed.count();
406
407     if (!isSorted(copyData))
408     {
409         cout << "Sorting error!" << endl;
410     }
411 }
412
413 return total / repeatTimes;
414 }
415
416 double measureWorstRandomTime(void (*sortFunction)(vector<int>&), int n, int permutationCount)
417 {
418     double maxTime = 0.0;
419
420     for (int i = 0; i < permutationCount; i++)
421     {
422         vector<int> data = generateRandomPermutation(n);
423
424         auto start = high_resolution_clock::now();
425
426         sortFunction(data);
427
428         auto end = high_resolution_clock::now();
429
430         duration<double, milli> elapsed = end - start;
431
432         if (elapsed.count() > maxTime)
433         {
434             maxTime = elapsed.count();

```

```
434         maxTime = elapsed.count();
435     }
436
437     if (!isSorted(data))
438     {
439         cout << "Sorting error!" << endl;
440     }
441 }
442
443 return maxTime;
444 }
445
446 void compositeSort(vector<int>& arr)
447 {
448     int n = arr.size();
449
450     if (n <= 100)
451     {
452         insertionSort(arr);
453     }
454     else if (n <= 2000)
455     {
456         quickSort(arr);
457     }
458     else if (n <= 4000)
459     {
460         mergeSort(arr);
461     }
462     else
463     {
464         heapSort(arr);
465     }
466 }
467
468 int main()
469 {
470     srand(time(0));
471
472     vector<int> sizes;
473
474     sizes.push_back(500);
475     sizes.push_back(1000);
476     sizes.push_back(2000);
477     sizes.push_back(3000);
```

```

468 int main()
469 {
470     srand(time(0));
471
472     vector<int> sizes;
473
474     sizes.push_back(500);
475     sizes.push_back(1000);
476     sizes.push_back(2000);
477     sizes.push_back(3000);
478     sizes.push_back(4000);
479     sizes.push_back(5000);
480
481     int repeatTimes = 5;
482     int permutationCount = 10;
483
484     cout << fixed << setprecision(4);
485
486     cout << "Clock used: C++ chrono high_resolution_clock" << endl;
487     cout << "Repeat times: " << repeatTimes << endl;
488     cout << "Random permutations: " << permutationCount << endl;
489     cout << endl;
490
491     cout << "===== Worst-Case Runtime Experiment =====" << endl;
492
493     cout << setw(8) << "n"
494         << setw(18) << "Insertion"
495         << setw(18) << "Quick"
496         << setw(18) << "Merge"
497         << setw(18) << "Heap"
498         << setw(18) << "Composite"
499         << endl;
500
501     cout << "-----" << endl;
502
503     for (int i = 0; i < sizes.size(); i++)
504     {
505         int n = sizes[i];
506
507         vector<int> insertionData = generateReverseData(n);
508         vector<int> mergeData = generateMergeWorstCaseData(n);
509         vector<int> compositeData = generateReverseData(n);

```

```

500
501     cout << "-----" << endl;
502
503     for (int i = 0; i < sizes.size(); i++)
504     {
505         int n = sizes[i];
506
507         vector<int> insertionData = generateReverseData(n);
508         vector<int> mergeData = generateMergeWorstCaseData(n);
509         vector<int> compositeData = generateReverseData(n);
510
511         double insertionTime = measureTime(insertionSort, insertionData, repeatTimes);
512         double quickTime = measureWorstRandomTime(quickSort, n, permutationCount);
513         double mergeTime = measureTime(mergeSort, mergeData, repeatTimes);
514         double heapTime = measureWorstRandomTime(heapSort, n, permutationCount);
515         double compositeTime = measureTime(compositeSort, compositeData, repeatTimes);
516
517         cout << setw(8) << n
518             << setw(18) << insertionTime
519             << setw(18) << quickTime
520             << setw(18) << mergeTime
521             << setw(18) << heapTime
522             << setw(18) << compositeTime
523             << endl;
524     }
525
526     cout << endl;
527     cout << "Unit: milliseconds" << endl;
528
529     return 0;
530 }

```


測試資料

Repeat times: 5
Random permutations: 10

===== Worst-Case Runtime Experiment =====					
n	Insertion	Quick	Merge	Heap	Composite
500	0.1278	0.0000	0.6362	0.0000	0.0000
1000	0.0000	0.0000	0.4066	0.4600	0.0000
2000	7.5004	0.0000	1.3468	2.8450	0.0000
3000	14.6378	2.0040	0.7206	0.0000	1.7148
4000	29.9296	0.0000	3.2832	6.9540	3.2354
5000	47.5894	5.4810	2.5382	10.8130	0.0000

Unit: milliseconds
PS C:\Users\tusno\Downloads\output>

效能分析:

時間複雜程度: $O(n^2 + k \cdot n \log n)$

空間複雜程度:

申論及開發報告: